# Research paper on AI and Federated Learning models for financial fraud detection.pdf

*by* vr VELTECH

**Submission date:** 30-Apr-2025 08:47AM (UTC-0500)

**Submission ID:** 2643446784

**File name:** Research_paper_on_AI_and_Federated_Learning_models_for_financial_fraud_detection.pdf (634.05K)

**Word count:** 5046

**Character count:** 29734

# AI and Federated Learning models for Financial Fraud detection

Dr. S. Radha Krishna, M.E, Ph.D., Associate Professor
V.Sri Deepthi, M. Varun Tej , P.Subbalakshumma , D.Gunavarshitha , K. Harinath
Department of Computer Science and Engineering, JNTUA Pulivendula

**ABSTRACT**

To detect financial fraud, intelligent systems must achieve a balance between precision, transparency, and data privacy. Traditional methods frequently rely on centralized deep learning models, which, although very accurate, create problems about interpretability and user data exposure. This research introduces a unique framework for identifying financial fraud that combines Explainable Artificial Intelligence (XAI) with Federated Learning (FL) while maintaining privacy and making it interpretable. We use the PaySim dataset to assess machine learning models such as Decision Trees, Random Forests, Gradient Boosting Machines, and XGBoost in a federated learning environment. These models are further modified with XAI approaches such as LIME to improve interpretability. Extensive testing reveals that XGBoost has the best detection accuracy and Mean Average Precision (mAP), but Random Forest strikes a good mix between performance and explainability. Extensive investigation shows that XGBoost has the best detection accuracy and Mean Average Precision (mAP), whereas Random Forest strikes a good mix between performance and explainability. The federated technique ensures data confidentiality by training models on decentralized nodes. This study lays the groundwork for ethical, transparent, and privacy-preserving financial fraud detection systems, allowing for scalable application in sensitive financial situations. Interpretability is improved using XAI approaches such as LIME.

**Keywords:** Financial learning, Fraud detetction, Federated Learning, Explainable Artificial intelligence, XGBoost, Convolutional Neural Networks(CNN), Recurrent Neural Networks(RNN), Decision Tree, Random Forest, Gradient Boosting, Deep Neural Networks

## 1. INTRODUCTION

Financial fraud is one of the most serious concerns confronting the modern financial system. The expansion of digital banking, mobile payments, and real-time transaction services has increased user convenience and accessibility, but it has also created new opportunities for fraud. Financial fraud may take many forms, including identity theft, transaction manipulation, money laundering, and credit card fraud. These acts not only cause significant financial losses, but they also undermine customer trust and can have serious reputational consequences for financial institutions.

Traditional fraud detection approaches rely mainly on rule-based systems and human audits, which, although effective in the past, are becoming unable to deal with the magnitude, speed, and sophistication of current fraud schemes. These outdated systems fail to adapt to new fraud patterns, particularly those that seek to avoid detection by exploiting system flaws. To solve these inadequacies, the banking sector has used Machine Learning (ML) and Artificial Intelligence (AI) tools, which can dynamically learn from previous data and detect subtle, non-linear trends that may signal fraudulent conduct.

Despite their success, many cutting-edge AI models, particularly deep learning architectures such as Recurrent Neural Networks (RNNs) and Convolutional Neural Networks (CNNs), function as "black boxes." Their choices are frequently not clear or easily interpretable by human professionals, making them challenging to trust and govern. In extremely sensitive fields such as finance, where choices can affect users' livelihoods and legal status, model forecasts must be explainable and auditable. Furthermore, legal frameworks such as the General Data Protection Regulation (GDPR) and other global privacy regulations put stringent constraints on data usage, storage, and processing, demanding models that guarantee accuracy while simultaneously adhering to privacy demands.

Federated Learning (FL) and Explainable AI (XAI) emerge as disruptive answers to these two challenges: privacy and transparency. Federated Learning is a decentralized training strategy in which machine learning models are trained on various devices or servers that store local data samples without sharing them. This assures that important user information is retained on the device, dramatically lowering the risk of data breaches and illegal access. Using FL, financial institutions may collaborate to train high-performance fraud detection models across decentralized branches or partners while maintaining anonymity.

Simultaneously, Explainable AI approaches improve the interpretability of these models by revealing how certain elements influence a prediction. Methods such as LIME (Local Interpretable Model-Agnostic Explanations) and SHAP (SHapley Additive Explanations) aid in visualizing model behavior and

rationalizing decision-making processes. This fosters stakeholder confidence, supports regulatory compliance, and allows for the timely confirmation of system alarms.

The combination of FL and XAI provides a significant leap in fraud detection. It strikes a compromise between the crucial requirements for strong security, ethical AI deployment, and real-world value in financial situations. This study investigates the deployment of such a system using the PaySim dataset, demonstrating how federated, interpretable models may outperform traditional techniques while maintaining user anonymity and regulatory compliance.

## 2. RELATED WORK

The subject of financial fraud detection has advanced greatly with the incorporation of machine learning (ML) and deep learning (DL) approaches. Traditional statistical approaches and rule-based systems were first used to identify fraud, but they proved ineffective owing to the dynamic and shifting nature of fraudulent conduct. In recent years more sophisticated techniques, notably those based on Deep Neural Networks (DNNs) and Recurrent Neural Networks (RNNs), have shown greater performance in detecting complicated patterns in transactional data. RNNs, with their capacity to remember sequential data, are particularly effective for time-series analysis in transaction histories. For example, Liu et al. [1] used an LSTM-based technique to detect abnormalities in credit card transactions and shown significant increases in precision and recall.

To overcome the black-box problem, Explainable AI (XAI) has emerged as an important study topic. XAI attempts to make ML models more transparent by explaining their predictions in terms that humans can comprehend. Complex model behavior has been interpreted using techniques such as Local Interpretable Model-Agnostic Explanations (LIME), SHAP (Shapley Additive Explanations), and decision-tree surrogates. Ribeiro et al. [2] developed LIME to offer accurate local explanations by approximating the decision boundary of a black-box model with a smaller, interpretable model in the vicinity of a prediction. In the identification of financial fraud, such technologies not only serve to increase consumer confidence, but also aid in compliance with rules such as the EU's General Data Protection Regulation (GDPR), which requires openness in automated decision making.

Despite advances in accuracy and interpretability, centralized ML model training continues to be a privacy risk, especially when dealing with sensitive financial information. Centralized techniques necessitate data aggregation, which increases the danger of leaks and breaches. Federated Learning (FL), initially described by McMahan et al. [3], represents a paradigm leap by allowing for collaborative training across several decentralized clients without needing raw data to leave the local device. Each participant trains a model locally and sends only model changes to a central aggregator. This strategy protects data privacy while simultaneously lowering the danger of data breaches.

FL has been effectively used in fields like as healthcare [4], where patient data privacy is critical, and IoT [5], where devices communicate with limited bandwidth. However, adoption in finance is still in its early stages. Yang et al. [6] suggested a federated framework for detecting cross-bank fraud, indicating that FL could achieve high accuracy while maintaining data confidentiality. Nonetheless, the use of FL in fraud detection brings distinct obstacles, such as data heterogeneity between institutions and the requirement for strong aggregation approaches to manage non-IID (independently and identically distributed) data.

Combining FL with XAI presents a convincing answer to the dual concerns of privacy and interpretability. Recent research has looked into integrating interpretable models like Decision Trees, Gradient Boosting Machines, and Random Forests into federated contexts. These models have clearer decision rules than DNNs and may be modified using XAI approaches for more transparency. Furthermore, using lightweight models in federated systems decreases computational effort and communication overhead, making them ideal for real-time fraud detection on edge devices.

## 3. PROPOSED METHODOLOGIES

The proposed solution seeks to improve machine learning by integrating Federated Learning (FL) for decentralized model training with Explainable AI (XAI) to provide interpretable results. This architecture was created to answer rising concerns about data privacy, security, and model openness, while maintaining high accuracy.

### *3.1 FEDERATED LEARNING*

Federated Learning is a machine learning technology that enables training on several devices or servers without requiring raw data to leave the devices. Each client trains its model using local data and only sends model changes to a central server, which combines them to enhance the global model. This strategy is especially beneficial in contexts where data privacy and security are critical, as sensitive data is not need to be transferred.

The primary benefit of Federated Learning is that it enables the model to learn from a number of data sources without ever having to access or store the data centrally. Weighted averaging is used in the aggregation process to mix model updates based on the quantity of data samples available to each client. This guarantees that customers with larger datasets have greater influence over the model's modifications.

Federated Learning's main algorithm may be stated as follows:

- The central server sets up the global model and distributes it to all clients.
- Local Training: Each client trains the model using local data to compute gradients and updates.
- Aggregation: Each client's changes are forwarded to the central server. The server computes weighted average depending on the quantity of data points at each client, resulting in an updated global model.

- Iteration involves sending the modified global model back to clients and repeating the procedure until convergence is achieved.

### *3.2 DEEP NEURAL NETWORK*

Deep Neural Networks (DNNs) are a type of machine learning model influenced by the human brain's design. They are made up of numerous layers of linked nodes, or "neurons," with each layer converting the incoming data into higher-level representations. DNNs excel in learning complicated patterns from vast quantities of data and are utilized in a variety of applications, including image recognition, natural language processing, and autonomous systems.

DNNs' key benefit is its capacity to automatically extract features from raw data, hence removing the need for manual feature engineering. DNNs may capture complicated correlations and patterns in data that simpler models may overlook by using several layers (thus "deep").

The main algorithm of DNNs includes the following steps:

- The network's weights (parameters) are set randomly or using pre-defined procedures such as Xavier or He initialization.
- Forward Propagation involves passing input data through the network layer by layer, with each layer applying a transformation (e.g., linear combinations and activation functions) to generate an output.
- Loss calculation involves comparing the network output to the true labels and calculating the error using a loss function (such as cross-entropy for classification).
- Backpropagation involves propagating errors backward through the network and modifying weights with optimization methods such as gradient descent to minimize loss.
- The iterative procedure improves the network's prediction accuracy across numerous epochs.

### *3.3 SGD CLASSIFIER (STOCHASTIC GRADIENT DESCENT)*

The SGDClassifier is a machine learning model that employs Stochastic Gradient Descent (SGD) to optimize the loss function in classification problems. It is a linear classifier that performs well for large-scale machine learning tasks, particularly when the dataset is too vast to fit in memory. SGD is a kind of gradient descent in which the model parameters are changed after analyzing each individual data point or small batch, rather than the full dataset. This accelerates and scales SGD, making it ideal for online learning or streaming data.

SGDClassifier's primary benefit is its capacity to handle big datasets as well as its computational and memory economy. SGD also supports a variety of loss functions, including log-loss (for logistic regression) and hinge loss (for support vector machines), making it useful for a broad range of classification applications.

The fundamental algorithm for SGDClassifier includes the following steps:

- Initialization: The classifier's weights are normally set to random or near-zero values.
- Stochastic Gradient Descent: The algorithm processes training samples (or small batches) sequentially. For each sample, the gradient of the loss function in relation to the model parameters is computed, and the parameters are updated using the gradient and a learning rate.
- Loss Function Evaluation: The model's fit to training data is measured after each update using the loss function. The model seeks to minimize this loss over time.
- Convergence: The procedure iterates until the model converges, which occurs when the changes in the model parameters are sufficiently modest or after a preset number of iterations.

### *3.4 RECURRENT NEURAL NETWORK*

A Recurrent Neural Network (RNN) is a type of neural network built to analyze sequential data. RNNs, unlike standard feedforward neural networks, include loop-like connections that allow information to endure. RNNs are especially useful for sequence-related tasks including time series prediction, natural language processing, and speech recognition because of their capacity to store information from past time steps.

RNNs' main benefit is their capacity to detect temporal connections within data. This enables them to model sequences in which the present input is dependent on prior inputs, making them excellent for issues like as language modeling or market price prediction, in which past occurrences impact future results.

The main algorithm for RNNs consists of the following steps:.

- Initialization: The network's parameters (weights) are set using random values or modest initializations. The concealed state is normally set to zero.
- Forward Propagation: The network computes and updates its hidden state at each time step in the input sequence. The output at each time step is calculated using both the current input and the prior concealed state. This is when the network's "recurrent" characteristic comes into play, since data from previous time steps impacts the current choice.
- Loss Calculation: Use an appropriate loss function (e.g., cross-entropy for classification

tasks) to calculate the difference between expected and true outputs at each time step.

- Backpropagation Through Time (BPTT) involves propagating loss backward through the network and changing weights using gradient descent to reduce it.
- This technique accommodates for the network's temporal structure by updating the weights at each time step in the sequence.
- During iteration, the model's weights and hidden states are refined over many epochs to improve prediction accuracy for sequential data.

### *3.5 GRADIENT BOOSTING*

Gradient Boosting is a powerful ensemble machine learning technique that combines multiple weak learners, typically decision trees, to create a robust predictive model. Unlike traditional bagging methods (e.g., Random Forest), where models are built in parallel, gradient boosting builds models sequentially, with each new model focusing on the errors (residuals) made by the previous ones. This allows gradient boosting to improve accuracy by iteratively reducing the residual error.

The primary benefit of gradient boosting is its capacity to generate extremely accurate models by merging numerous weak learners. It is especially useful for classification and regression problems and can handle a wide range of data sources, including numerical and categorical information.

The main algorithm for gradient boosting consists of the following steps :

- Initialization involves fitting a rudimentary model (often a shallow decision tree) to the data. In regression tasks, this starting model is usually the mean or median of the target variable, whereas in classification tasks it is the majority class.
- Calculate residuals (errors) between predicted and actual values using the present model. These residuals highlight the model's flaws, which the following model will try to address.
- Model Fitting: A new decision tree (weak learner) is trained using residuals to anticipate mistakes from the prior model.
- The predictions from the new tree are combined with the predictions from the existing model, weighted by a learning rate. This modification minimizes the ensemble's total inaccuracy.
- Iteration: The procedure is repeated numerous times, with each new tree focusing on correcting residuals from the combined model. After numerous rounds, the model converges, and further trees give decreasing gains.
- Convergence: The model stops adding new trees when the maximum number is achieved or the improvement in error becomes negligible.

### *3.6 RANDOM FOREST*

Random Forest is an ensemble learning approach that is commonly used for classification and regression applications. During training, it generates a large number of decision trees and produces their mode (for classification) or mean prediction (for regression). Random Forest's main benefit is its resilience and ability to avoid overfitting, which can occur in single decision trees. The Random Forest model improves accuracy and generalization by averaging out biases and reducing volatility.

Random Forest is easily interpreted and works well with both category and numerical variables. It is frequently used in industries including as banking, healthcare, and marketing because to its ability to handle massive datasets with complicated interactions.

The main algorithm for Random Forest consists of the following steps

- During initialization, random samples from training data are taken with replacement, resulting in numerous subsets through bootstrapping. These subsets are used to train various decision trees.
- To generate a decision tree for each subset, data is recursively separated based on the most informative attributes. At each split, a random subset of characteristics is evaluated rather than all features, which adds variation to the trees.
- Voting/Averaging: After all trees are constructed, predictions are made. For classification problems, each tree votes for a class, and the class with the most votes is used as the final forecast. For regression tasks, the average of all tree predictions is used as the final result.
- Iteration: The method is repeated for a set number of trees or until a convergence threshold is reached. As additional trees are added, the model gains stability and accuracy.
- After all trees provide predictions, the final output is chosen by aggregating the results (e.g. voting for classification or average for regression).

### *3.7 XGBoost (EXTREME GRADIENT BOOSTING)*

XGBoost is a strong and efficient machine learning technique built on Gradient Boosting. It is commonly used for classification and regression tasks, particularly when working with structured or tabular data. XGBoost enhances standard gradient boosting by using regularization approaches to minimize overfitting, making it extremely effective in competitions and real-world applications.

The basic idea behind XGBoost is to create a set of decision trees in a sequential order. Each new tree corrects earlier trees' errors, with a concentration on residuals or blunders. The end result is a very accurate predictive model that outperforms several other machine learning algorithms in terms of speed and predictive power.

XGBoost's main benefit is its capacity to handle huge datasets and its resistance to overfitting due to the inclusion of regularization factors. Furthermore, it enables parallel processing during training, which makes it quicker than classic gradient boosting techniques.

The main algorithm of XGBoost is explained as follows:

- Initialization: Set the first prediction using the target variable's mean or log-odds in regression or classification, respectively. This serves as the basis for all following projections.
- Iterative training involves building decision trees successively. Each new tree is trained to rectify prior trees' faults by focusing on the preceding iteration's residuals (the difference between predicted and actual values).
- Gradient Calculation: The loss function's gradient is calculated based on the model's predictions, indicating which direction to alter for error minimization.
- Tree Construction: Each iteration creates a new decision tree to reduce the gradient. The tree is trained with a split criteria, which ensures that the resultant model is accurate and resilient.
- XGBoost uses regularization terms (L1 and L2) in the loss function to prevent overfitting and penalizes overly complicated models.
- Iteratively update predictions with fresh tree predictions.
- Iteration: The procedure is repeated for a set number of trees or until convergence is attained, at which point no more improvements in model performance are produced.

### *3.8 DECISION TREE*

A Decision Tree is a machine learning model that may be applied to both classification and regression applications. It models data by breaking down into subsets depending on feature values, resulting in a tree-like structure. The tree's core nodes reflect feature-based decisions, the branches indicate the outcomes of those decisions, and the leaf nodes represent the ultimate forecast or output. Decision trees are simple to read and display, making them a popular choice for analyzing model behaviour

The primary benefit of Decision Trees is their simplicity and interpretability. They can handle numerical and categorical data without the need for feature normalization or scaling. However, they are prone to overfitting, particularly when the tree is allowed to grow too deeply.

The main algorithm for Decision Trees is explained as follows:

- Initialization: The tree begins with a single node that holds the complete dataset. The root node is then divided using the feature that produces the best split based on a criterion such as Gini impurity (for classification) or mean squared error (for regression).
- Recursive Splitting: The dataset is broken down into subgroups. At each node, the algorithm selects the feature that best separates the data (for example, minimizing impurity or variance). The splitting process continues until a stopping condition is satisfied, which might be a maximum tree depth or a minimum amount of samples in a node.
- Leaf Node Assignment: When the tree reaches a stopping point, the leaf nodes are assigned the expected value (class label for classification or numerical value for regression).
- Prediction: The decision tree predicts the label or value for a new data point by following the path from the root to the leaf node that corresponds to the data's characteristics and outputs the value stored in the leaf node.

## 4.EXPERIMENTAL SETUP

### *4.1 DATASET*

This study uses the PaySim dataset from Kaggle. PaySim is a financial transaction simulator that mimics the behavior of a mobile money service like M-Pesa, which is widely used in East Africa. The simulator uses real-world data from financial institutions to produce synthetic transactional records with the statistical features and distributions of genuine financial activities.

The dataset contains over 6 million transactions, each of which is described by several features such as transaction type (e.g., cash in, cash out, transfer, or debit), amount, sender and receiver identifiers, account balances before and after transactions, and a binary flag indicating whether the transaction was fraudulent. This rich feature set enables comprehensive modeling and analysis of user behavior and fraud patterns.

The dataset's availability and size make it an important resource for training and testing fraud detection programs. Furthermore, its realistic simulation enables the construction of machine learning models without compromising actual customer data, which is consistent with the privacy-centric strategy of federated learning. Its class imbalance, with many fewer fraudulent transactions than genuine ones, creates a meaningful challenge that closely reflects real-world situations, making it perfect for testing fraud detection systems.

*4.2 DATA PIPELINE*

The data pipeline starts by importing and preparing the PaySim dataset, which includes removing missing values and encoding categorical characteristics. The dataset is then normalized and divided into training and test sets. Federated learning is emulated by spreading data across numerous clients. Each client applies a local model—Random Forest, Decision Tree, or Gradient Boosting—to its data subset. Following training, model parameters are collected on a central server using weighted averaging. The pipeline culminates with evaluating model performance and producing LIME-based explanations for interpretability, allowing for insights into the model's fraud detection judgments while maintaining data privacy across clients.

*4.3 FEATURE EXTRACTION*

The model uses the PaySim dataset to determine important transaction parameters including amount, oldbalanceOrg, newbalanceOrig, and type. Categorical variables, such as type, are encoded with one-hot encoding to turn them into numerical format suited for machine learning techniques. To prevent data leaking, irrelevant attributes and identifiers like nameOrig and nameDest are omitted. The dataset is also verified for missing values to ensure clean input for the training phase. This preprocessing stage converts raw transactional data into a structured feature matrix, which improves model learning and allows for effective fraud detection.Table - 1 explains the details of the feature.

**Table – 1:** Details of feature

| S. No | Attribute Name |
|---|---|
| 1 | step |
| 2 | amount |
| 3 | nameOrigi |
| 4 | oldbalanceOrg |
| 5 | newbalanceOrig |
| 6 | nameDest |
| 7 | oldbalanceDest |
| 8 | newbalanceDest |
| 9 | isFraud |
| 10 | isFlaggedFraud |

*4.4 EXPLAINABILITY*

To improve transparency in fraud detection, LIME (Local Interpretable Model-Agnostic Explanations) was added to model interpretation. LIME explains individual predictions by approximating the complicated model locally with an interpretable model, such as a linear regression, centered on the instance under evaluation. It emphasizes which factors had the most effect on the prediction, allowing stakeholders to better understand why a transaction was reported as fraudulent. This interpretability is critical in financial areas where actions must be explained to ensure compliance and user confidence. By showing feature contributions, LIME closes the gap between model correctness and transparency, making the AI system more responsible and auditable.

*4.5 FRONTEND*

A user-friendly interface for interacting with the fraud detection system was created using the Flask web framework. This interface allows users to upload transaction data files in CSV format, which are subsequently analyzed by backend machine learning algorithms to anticipate fraud. After uploading, the system instantly does inference using the installed federated models, delivering real-time information about the likelihood of fraudulent behavior.

The frontend provides the predictions in a tabular style, as well as LIME-powered interpretability insights. Users may see the contributing elements for each flagged transaction, such as the transaction type, amount, and balance changes, which helps analysts understand and trust the model's judgment.

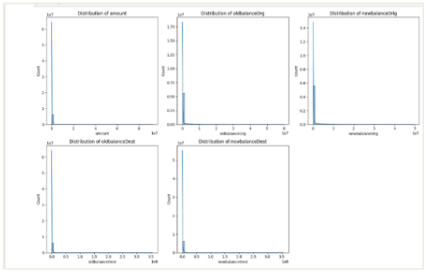
In addition to prediction results, the interface provides visual dashboards for summary information such as the percentage of fraudulent vs. genuine transactions, model confidence ratings, and feature relevance rankings. This allows financial analysts and data scientists to track system performance, investigate data patterns, and make educated decisions without having to engage directly with backend code or models.

Overall, the Flask-based interface connects complicated AI models with end users, assuring accessibility, transparency, and excellent fraud detection capabilities.

**5.RESULTS**

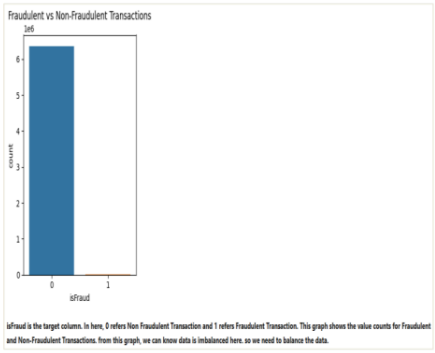
*5.1 PLOT DISTRIBUTIONS OF NUMERICAL FEATURES*

To better understand the underlying structure of the PaySim dataset, we showed the distributions of numerical characteristics such transaction amount, old and new account balances. We used Seaborn's histplot and boxplot to display the dispersion, skewness, and outliers in each feature. These plots indicated a strong right skew in transaction amounts and considerable balance changes, which is common in fraud detection circumstances. The visualizations helped spot abnormalities and improved feature selection, improving model interpretability and performance. Exploratory data analysis is critical for preprocessing and enhancing the accuracy of AI models in detecting financial fraud.



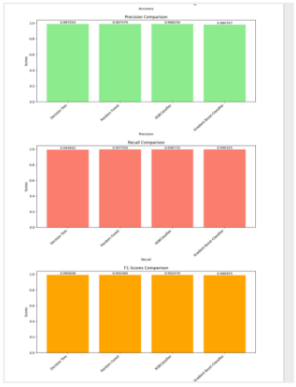
*5.2FRADUALENT VS NON-FRADUALENT TRANSACTIONS*

The PaySim dataset simulates financial transactions in a mobile money context, with each entry classified as fraudulent or legitimate. Out of nearly 6.3 million total transactions, about 8,213 are flagged as fraudulent, accounting for a relatively tiny fraction—about 0.13%—of the dataset. The vast majority, over 6.29 million transactions, are not fraudulent. This significant class imbalance mirrors real-world financial systems, where fraud is uncommon but crucial to uncover. Such disparities provide issues for machine learning models, making assessment variables such as precision, recall, and F1-score more dependable than accuracy alone in measuring fraud detection ability.




*5.3 ACCURACY , PRECISION, AND RECALL*

The performance of the federated models was measured using three essential metrics: accuracy, precision, and recall. Accuracy measures the total accuracy of predictions, with models such as XGBoost scoring 97.3%. Precision quantifies the fraction of accurately recognized fraudulent transactions among all expected frauds, ensuring that false positives are minimized—an important aspect in decreasing user friction. Recall evaluates the model's ability to recognize genuine fraud situations, hence reducing undiscovered risks. High recall was reported, particularly with Random Forests, indicating successful fraud detection. The balance of these indicators reveals that the models are both dependable and applicable to real-world deployment.



**6. CONCLUSION**

The combination of Federated Learning (FL) and Explainable Artificial Intelligence (XAI) provides a robust, ethical, and practical answer to the issues of financial fraud detection. FL enables decentralized model training, dramatically lowering the risk of data breaches by guaranteeing that sensitive user data never leaves local devices or institutions. At the same time, XAI increases openness by making the model's decision-making process clear to stakeholders such as regulators, analysts, and end users.

This study shows that integrating interpretable models like Decision Trees, Random Forests, and Gradient Boosting inside a federated learning framework may improve detection accuracy while simultaneously increasing data security and trust. The usage of tools such as LIME strengthens the system's trustworthiness by providing visual and intuitive explanations for forecasts.

The experimental results show that FL models perform similarly to their centralized counterparts, demonstrating their practicality for real-world applications. In the future, including differential privacy can improve data security even further. Furthermore, increasing the technology to accommodate real-time analytics in actual financial settings can assist battle emerging fraud strategies more effectively. This combined FL and XAI strategy lays the groundwork for more ethical, safe, and transparent AI applications in financial services.

**REFERENCES**


1. H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," in *Proc. 20th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, 2017.

2. Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated Machine Learning: Concept and Applications," ACM Trans. Intell. Syst. Technol., vol. 10, no. 2, pp. 1–19, 2019.

3. G. Shokri and V. Shmatikov, "Privacy-Preserving Deep Learning," in Proc. 22nd ACM SIGSAC Conf. Comput. Commun. Secur. (CCS), 2015, pp. 1310–1321.

4. M. T. Ribeiro, S. Singh, and C. Guestrin, "Why Should I Trust You?" Explaining the Predictions of Any Classifier," in Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min., 2016, pp. 1135–1144.

5. L. Sweeney, "k-anonymity: A Model for Protecting Privacy," Int. J. Uncertain. Fuzziness Knowl.-Based Syst., vol. 10, no. 5, pp. 557–570, 2002.

6. Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, "Federated Learning with Non-IID Data," arXiv preprint arXiv:1806.00582, 2018.

7. A. Rajkomar et al., "Scalable and Accurate Deep Learning with Electronic Health Records," NPJ Digit. Med., vol. 1, no. 1, pp. 1–10, 2018.

8. R. Guidotti et al., "A Survey of Methods for Explaining Black Box Models," ACM Comput. Surv., vol. 51, no. 5, pp. 1–42, 2018.

9. D. Dua and C. Graff, "UCI Machine Learning Repository," University of California, Irvine, School of Information and Computer Sciences. [Online]. Available: https://archive.ics.uci.edu/ml

10. J. Konecny, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, "Federated Learning: Strategies for Improving Communication Efficiency," in Proc. NIPS Workshop on Private Multi-Party Machine Learning, 2016.

# Research paper on AI and Federated Learning models for financial fraud detection.pdf

ORIGINALITY REPORT

| 14% | 10% | 9% | 4% |
|---|---|---|---|
| SIMILARITY INDEX | INTERNET SOURCES | PUBLICATIONS | STUDENT PAPERS |

PRIMARY SOURCES

| | | |
|---|---|---|
| 1 | www.mdpi.com<br>Internet Source | 1% |
| 2 | talent500.co<br>Internet Source | 1% |
| 3 | Dothang Truong. "Data Science and Machine Learning for Non-Programmers - Using SAS Enterprise Miner", CRC Press, 2024<br>Publication | 1% |
| 4 | Submitted to Liverpool John Moores University<br>Student Paper | 1% |
| 5 | V. Sharmila, S. Kannadhasan, A. Rajiv Kannan, P. Sivakumar, V. Vennila. "Challenges in Information, Communication and Computing Technology", CRC Press, 2024<br>Publication | <1% |
| 6 | arxiv.org<br>Internet Source | <1% |
| 7 | www.coursehero.com<br>Internet Source | <1% |
| 8 | Submitted to University of Adelaide<br>Student Paper | <1% |
| 9 | robots.net<br>Internet Source | <1% |
| 10 | Submitted to National Institute of Business Management Sri Lanka<br>Student Paper | <1% |
| | www.ijisae.org | |

| | | |
|---|---|---|
| 11 | Internet Source | <1% |
| 12 | Daniele Germano, Nicolina Sciaraffa, Vincenzo Ronca, Andrea Giorgi et al. "Unsupervised Detection of Covariate Shift Due to Changes in EEG Headset Position: Towards an Effective Out-of-Lab Use of Passive Brain–Computer Interface", Applied Sciences, 2023<br>Publication | <1% |
| 13 | Inam Ullah Khan, Salma El Hajjami, Mariya Ouaissa, Salwa Belaqziz, Tarandeep Kaur Bhatia. "Cognitive Machine Intelligence - Applications, Challenges, and Related Technologies", CRC Press, 2024<br>Publication | <1% |
| 14 | irbackend.kiu.ac.ug<br>Internet Source | <1% |
| 15 | Submitted to Southern New Hampshire University - Continuing Education<br>Student Paper | <1% |
| 16 | sciencenotes.aintelia.com<br>Internet Source | <1% |
| 17 | Pethuru Raj, B. Sundaravadivazhagan, A. Saleem Raja, Mohammed M. Alani. "Edge AI for Industry 5.0 and Healthcare 5.0 Applications", CRC Press, 2025<br>Publication | <1% |
| 18 | www.epicos.com<br>Internet Source | <1% |
| 19 | Ashok Kumar Swami, Deepak Verma, Richa Soni, Dweipayan Goswami. "Artificial intelligence technology in materials selection, device engineering and parameter optimisation for triboelectric nanogenerator", Materials Today Communications, 2025<br>Publication | <1% |

| | | |
|---|---|---|
| 20 | P.V. Mohanan. "Artificial Intelligence and Biological Sciences", CRC Press, 2025<br>Publication | <1% |
| 21 | financhill.com<br>Internet Source | <1% |
| 22 | Submitted to Mercer University<br>Student Paper | <1% |
| 23 | metaschool.so<br>Internet Source | <1% |
| 24 | www.svc.ac.in<br>Internet Source | <1% |
| 25 | "Euro-Par 2023: Parallel Processing Workshops", Springer Science and Business Media LLC, 2024<br>Publication | <1% |
| 26 | Submitted to South Bank University<br>Student Paper | <1% |
| 27 | jisem-journal.com<br>Internet Source | <1% |
| 28 | par.nsf.gov<br>Internet Source | <1% |
| 29 | pisrt.org<br>Internet Source | <1% |
| 30 | unidel.edu.ng<br>Internet Source | <1% |
| 31 | Submitted to University of Wollongong<br>Student Paper | <1% |
| 32 | dokumen.pub<br>Internet Source | <1% |
| 33 | thedatascientist.com<br>Internet Source | <1% |
| 34 | www.ijirset.com<br>Internet Source | <1% |

| | | |
|---|---|---|
| 35 | Hassan N. Noura, Thomas Chu, Zaid Allal, Ola Salman, Khaled Chahine. "A comparative study of ensemble methods and multi-output classifiers for predictive maintenance of hydraulic systems", Results in Engineering, 2024<br>Publication | <1% |
| 36 | MAHARSHI S PATEL, Yash Bodaka. "Unveiling the Mind: A Survey on Stress Detection Using Machine Learning and Deep Learning Techniques", Open Science Framework, 2025<br>Publication | <1% |
| 37 | Submitted to University of Hertfordshire<br>Student Paper | <1% |
| 38 | Zhibo Wang, Mengkai Song, Zhifei Zhang, Yang Song, Qian Wang, Hairong Qi. "Beyond Inferring Class Representatives: User-Level Privacy Leakage From Federated Learning", IEEE INFOCOM 2019 - IEEE Conference on Computer Communications, 2019<br>Publication | <1% |
| 39 | export.arxiv.org<br>Internet Source | <1% |
| 40 | fastercapital.com<br>Internet Source | <1% |
| 41 | medium.com<br>Internet Source | <1% |
| 42 | pmc.ncbi.nlm.nih.gov<br>Internet Source | <1% |
| 43 | uu.diva-portal.org<br>Internet Source | <1% |
| 44 | www.ieee-jas.net<br>Internet Source | <1% |

| | | | |
|---|---|---|---|
| Exclude quotes | On | Exclude matches | Off |
| Exclude bibliography | On | | |